

# [11] Milvus: A Purpose-Built Vector Data Management System

저자: Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu 외 다수 (Zilliz, HUST)

학회/년도: SIGMOD 2021

분량: 약 14 페이지

역할군: (C) 경쟁/비교 시스템

---

## 요약

Milvus 는 **전문(특화) 벡터 데이터베이스의 대표 시스템**으로, 벡터 데이터의 저장과 대규모 유사도 검색에 특화되어 설계되었다. 관계형 데이터베이스와 달리, 복잡한 SQL 쿼리 기능보다 **벡터 검색 성능과 확장성**에 집중한다.

Milvus 의 핵심 특징은:

1. **클라우드 네이티브 아키텍처**: 저장과 연산의 분리로 탄력적 확장 가능
2. **세그먼트 기반 관리**: 데이터를 불변 세그먼트로 관리하여 인덱싱 간소화
3. **다양한 벡터 인덱스 지원**: FLAT, IVF\_FLAT, IVF\_PQ, HNSW, DiskANN 등
4. **조절 가능한 일관성**: 성능과 데이터 일관성의 트레이드오프를 사용자가 제어
5. **실제 산업 적용**: 이미지/비디오 검색, 추천 시스템, 생체 인증 등 10+ 실제 응용

Milvus 는 높은 검색 성능과 확장성을 제공하지만, **멀티 테이블 조인**이나 **복잡한 SQL 분석 쿼리**를 지원하지 않는다는 근본적 한계가 있다. 이것이 Exqutor 가 범용 벡터 DB(pgvector, VBASE, DuckDB)를 대상으로 하는 이유이다.

---



## 상세분석

### 4.1 전문 벡터 DB 란 무엇인가

전문 벡터 DB 는 **벡터 데이터의 저장과 유사도 검색**에 특화된 시스템이다. 관계형 데이터베이스처럼 복잡한 SQL 쿼리를 지원하는 것이 목표가 아니라, 대규모 벡터를 빠르고 정확하게 검색하는 것이 목표이다.

Milvus 가 해결하는 핵심 도전 과제 5 가지:

1. **사용하기 쉬운 인터페이스**: 다양한 응용 분야의 개발자가 쉽게 벡터 검색을 활용할 수 있도록 파이썬, Java, Go 등 여러 언어의 SDK 를 제공한다. 개발자는 복잡한 인덱스 구조를 이해할 필요 없이, 간단한 API(search, insert, delete)만으로 벡터 검색 시스템을 구축할 수 있다.
2. **이기종 컴퓨팅 최적화**: CPU 와 GPU 를 효율적으로 활용하여 성능을 극대화한다. 벡터 검색은 높은 병렬성을 가지므로, GPU 의 수천 개 코어를 활용하면 성능을 10 배 이상 향상시킬 수 있다. Milvus 는 이러한 GPU 가속을 자동으로 처리하므로, 개발자는 GPU 프로그래밍 없이도 GPU 의 이점을 누릴 수 있다.
3. **단순 유사도 검색을 넘어선 고급 쿼리**: 속성 필터링(메타데이터 조건), 다중 벡터 검색(여러 벡터 필드 동시 검색), 범위 검색(거리 임계값) 등 실무에서 필요한 고급 기능을 지원한다.
4. **동적 데이터 관리**: 실시간으로 데이터가 삽입·수정·삭제되면서 동시에 검색 요청을 처리해야 하는 상황을 안정적으로 지원한다. 인덱스를 재구축하지 않으면서도 새로운 데이터를 즉시 검색 가능하게 만드는 것이 핵심이다.
5. **확장성과 가용성**: 분산 환경에서 데이터와 쿼리 처리를 여러 노드에 분산시켜, 수십억 건의 벡터를 저장·검색할 수 있어야 한다. 동시에 노드 장애 시에도 서비스가 중단되지 않도록 복제(replication)를 지원해야 한다.

## 4.2 시스템 구조: 클라우드 네이티브 아키텍처

Milvus 는 클라우드 네이티브 아키텍처를 채택하여 다음 특징을 갖는다.

### 저장과 연산의 분리:

- **저장소:** Amazon S3, Google Cloud Storage 같은 객체 스토리지를 사용하여 고가용성과 확장성을 확보한다. 데이터는 스토리지에 영구적으로 저장되며, 여러 노드에서 동시에 접근할 수 있다.
- **연산:** 상태 없는(stateless) 컴포넌트들이 탄력적으로 확장·축소된다. 쿼리 처리 노드는 필요에 따라 동적으로 추가하거나 제거할 수 있으므로, 트래픽 변동에 자동으로 대응할 수 있다.

### 세그먼트 기반 데이터 관리:

- 데이터는 **컬렉션(Collection)** 단위로 관리되며, 이는 관계형 DB 의 테이블에 해당한다.
- 컬렉션은 여러 **세그먼트(Segment)**로 나뉘며, 이는 물리적 저장 단위이다. 각 세그먼트는 불변(immutable)이므로, 한 번 생성되면 수정되지 않는다.
- 세그먼트 단위로 독립적인 인덱스를 구축하고 검색을 수행한다. 새 데이터가 들어오면 새 세그먼트가 생성되고, 백그라운드에서 정기적으로 여러 세그먼트를 병합한다.

### 조절 가능한 일관성(Tunable Consistency):

- **로그 백본(log backbone):** 모든 쓰기 작업을 순서대로 로그에 기록하여, 분산 시스템에서 데이터 일관성을 관리한다.
- 사용자는 일관성-성능 트레이드오프를 제어할 수 있다:
  - **강한 일관성(Strong):** 쓰기 직후 읽기 시 최신 데이터 보장. 하지만 모든 노드가 동기화될 때까지 기다려야 하므로 느림.
  - **세션 일관성(Session):** 같은 세션 내에서만 읽기 일관성 보장. 서로 다른 세션은 약간의 시간 차이가 있을 수 있음.
  - **최종 일관성(Eventual):** 가장 빠르지만, 최신 데이터 반영에 약간의 지연이 있을 수 있음.

### 4.3 지원하는 벡터 인덱스: 다양한 시나리오 대응

Milvus 는 다양한 시나리오에 맞는 벡터 인덱스를 지원한다:

| 인덱스 유형   | 특징                  | 적합한 경우                          |
|----------|---------------------|---------------------------------|
| FLAT     | 전수 검색 (brute-force) | 소규모 데이터(~10 만 건), 100% 정확도 필요 시 |
| IVF_FLAT | 클러스터링 기반 + 전수 검색    | 중규모 데이터(~100 만 건), 높은 정확도 원할 때  |
| IVF_PQ   | 클러스터링 + 양자화(압축)     | 대규모 데이터(~수억 건), 메모리 절약 필수 시     |
| HNSW     | 계층적 그래프 기반          | 빠른 검색 필요, 메모리 충분할 때             |
| SCANN    | Google 개발, 양자화 기반   | 높은 처리량, 실시간 검색 필요할 때            |
| DiskANN  | SSD 기반              | 메모리 부족 시 대규모 데이터 저장 필요할 때       |

인덱스 선택의 실제 고려사항:

- **데이터 크기:** 작을수록 정확도 높은 FLAT/IVF\_FLAT 선택. 클수록 성능 중심의 PQ/HNSW 선택.
- **응답 시간:** 밀리초 단위 응답이 필요하면 HNSW. 수초 단위 허용하면 IVF\_PQ 로 메모리 절약.
- **업데이트 빈도:** 자주 업데이트되면 HNSW(동적 구조). 거의 업데이트 안 되면 IVF\_PQ(정적 구조) 선택.
- **정확도 요구사항:** 재현율(recall) 99%+ 필요하면 HNSW. 90% 정도 허용하면 PQ.

## 4.4 고급 쿼리 기능: 벡터 검색을 넘어서

Milvus 는 단순 top-k 를 넘어서 다양한 고급 기능을 지원한다.

**속성 필터링:** 벡터 유사도 검색 시 스칼라 속성 조건을 함께 적용

```
results = collection.search(
    data=[query_vector],
    anns_field="embedding",
    param={"metric_type": "L2", "params": {"nprobe": 10}},
    limit=10,
    expr="price < 100 AND category == 'electronics'" # 속성 필터
)
```

이렇게 하면 임베딩이 유사하면서 동시에 가격이 100 이하이고 카테고리가 전자제품인 상품만 반환된다.

**다중 벡터 검색:** 여러 벡터 필드를 동시에 검색하고 결과를 결합. 예를 들어, 상품의 이미지 벡터와 설명 텍스트 벡터를 모두 고려하여 유사한 상품을 찾을 수 있다.

**범위 검색:** 거리 임계값을 기반으로 한 범위 검색. "매우 유사한(거리 < 0.1) 데이터만" 찾아달라는 식의 쿼리를 지원한다.

## 4.5 실제 응용 사례: 10+ 산업 분야

논문은 Milvus 위에 구축한 실제 응용을 소개한다:

- **이미지/비디오 검색 시스템:** 전자상거래 플랫폼에서 "유사한 상품 찾기", SNS 에서 "같은 사진의 다른 게시물 찾기"
- **화학 분자 구조 분석:** 신약 개발 과정에서 기존 화학식과 유사한 새로운 화학식 검색
- **COVID-19 데이터셋 검색:** 전염병 연구에서 유사한 사례 검색
- **개인화 추천 시스템:** 사용자 행동 벡터와 상품 벡터의 유사도를 기반으로 추천
- **생체 인증:** 얼굴 인식, 홍채 인식 등에서 데이터베이스 내 유사한 생체 정보 검색
- **지능형 질의응답(Q&A):** 사용자 질문의 의미 벡터와 기존 답변의 의미 벡터 비교

## 4.6 본 논문과의 관계: 전문 vs. 범용의 핵심 차이

Milvus 같은 전문 벡터 DB의 **근본적 한계**는 복잡한 관계형 연산(조인, 집계, 서브쿼리 등)을 지원하지 않는다는 것이다.

**속성 필터링의 한계:** Milvus의 속성 필터링은 단일 컬렉션 내에서만 작동한다. 예를 들어:

- 가능: "가격 < 100 AND 카테고리 == 전자제품인 유사 상품 찾기"
- 불가능: "유사 상품을 찾되, 해당 상품을 판매하는 판매자의 등급이 4 점 이상인 경우만" (판매자 정보는 다른 컬렉션에 있음)

**조인 불가능:** 두 벡터를 기반으로 두 테이블을 조인할 수 없다.

이것이 바로 Exqutor가 Milvus 대신 pgvector, VBASE, DuckDB 같은 **범용 벡터 DB**를 대상으로 하는 이유이다. 범용 벡터 DB는 SQL의 모든 기능을 지원하지만, 벡터 검색의 카디널리티 추정이 부정확하다는 문제가 있다. Exqutor는 이 문제를 해결한다.

---



## 추가 제기 문제

Milvus 의 세그먼트 기반 아키텍처는 확장성은 뛰어나지만, 세그먼트 수가 많아지면 쿼리 시 모든 세그먼트를 방문해야 하는 오버헤드가 발생할 수 있다. 이를 해결하기 위해 백그라운드에서 주기적으로 세그먼트를 병합하는 정책이 필요하지만, 병합과 검색 사이의 타이밍을 어떻게 최적화할 것인가는 여전히 설계 문제로 남아있다.

또한 Milvus 가 속성 필터링을 지원하더라도, 필터의 선택도에 따라 인덱스 접근 전략이 어떻게 최적화되는지에 대한 공개 정보가 제한적이다. Exqutor 의 ECQO 개념을 Milvus 에 적용할 수 있을까에 대한 고민은 흥미로운 미래 연구 주제이다.